Lösungen der Warm-up-Karten

Gleiche Nummerierung wie auf den Karten. Fachlich richtige Alternativen sind gleichwertig – entscheidend sind die genannten Kernpunkte.

W6-01 Die Welle bricht · Fehler im Java-Code

a) schadenOben(treffer, 2) liefert 12 + 7 = 19. Laufzeitfehler z. B. bei schadenOben(treffer, 4): Nach drei pop-Aufrufen ist der Stapel leer, das vierte pop() löst eine IllegalStateException aus. Die Schleife zählt nur bis n und fragt nie, ob der Stapel leer ist.

b) ersterUeber(welle, 30) liefert 1 (25 ≤ 30, 40 > 30). Laufzeitfehler z. B. bei ersterUeber(welle, 90): Kein Monster hat mehr als 90 Lebenspunkte, i wird 3 und getItem(3) löst eine IndexOutOfBoundsException aus (gültig sind nur 0 bis getLength() − 1). Dasselbe passiert bei jeder leeren Welle. Außerdem wird −1 nie zurückgegeben (Vorgabe verfehlt, P3).

c) siehe Code. Abi-Check: vor jedem pop/top/dequeue/head die Leer-Abfrage, vor jedem getItem die Indexgrenze 0 … getLength() − 1. Mit der Korrektur: schadenOben(treffer, 4) = 49, ersterUeber(welle, 90) = −1.

```
public static int schadenOben(Stack treffer, int n) {
    int summe = 0;
    int i = 0;
    while (i < n && !treffer.isEmpty()) {
        summe = summe + (Integer) treffer.pop();
        i++;
    }
    return summe;
}
```

```
public static int ersterUeber(DynArray welle, int grenze) {
    for (int i = 0; i < welle.getLength(); i++) {
        Monster m = (Monster) welle.getItem(i);
        if (m.getLeben() > grenze) {
            return i;
        }
    }
    return -1;
}
```

W6-02 Die Welle bleibt stehen · Methode implementieren

a) siehe Code: Jedes Monster wird genau einmal entnommen, verglichen und in die Hilfsschlange eingereiht; danach wandern alle zurück. Eine Schlange als Hilfsstruktur erhält die Reihenfolge. Die Rückgabe steht erst nach der Wiederherstellung (ein return in der ersten Schleife würde q zerstört zurücklassen).

b) Wyvern: Stärke 14 ist die größte; Mimik hat ebenfalls 14, steht aber weiter hinten – wegen des strengen > bleibt das vordere Monster.

c) q enthält danach (vorn zuerst) Golem, Mimik, Wyvern, Ghul – die Reihenfolge ist umgekehrt, weil ein Stapel das zuletzt abgelegte Monster zuerst herausgibt. „Unverändert“ heißt: dieselben Objekte in derselben Reihenfolge. Mit Hilfsstapel bräuchte man einen zweiten Umweg (über einen weiteren Stapel) – einfacher ist die Hilfsschlange.

```
public static Monster staerkstes(Queue q) {
    Monster bestes = null;
    Queue hilf = new Queue();
    while (!q.isEmpty()) {
        Monster m = (Monster) q.dequeue();
        if (bestes == null || m.getStaerke() > bestes.getStaerke()) {
            bestes = m;
        }
        hilf.enqueue(m);
    }
    while (!hilf.isEmpty()) {
        q.enqueue(hilf.dequeue());
    }
    return bestes;
}
```

W6-03 Falsch beschriftete Zauberformeln · Methodenkopf und Signatur

a) (1) Parameter muss eine Reihung sein: Gegenstand[] schatz. (2) Rückgabetyp Kommazahl → double (mit int ginge beim Teilen der Nachkommaanteil verloren, vgl. P4). (3) Parameter und Rückgabetyp vertauscht: Parameter Queue, Rückgabe DynArray. (4) static fehlt (Klassenoperation) und Rückgabetyp Wahrheitswert → boolean statt void. Richtige Köpfe siehe Code.

b) Der Compiler meldet „missing return statement“: Ist keine der beiden Bedingungen erfüllt, erreicht der Ablauf das Ende der Methode ohne return. In jedem möglichen Pfad muss eine Rückgabe stehen – Korrektur: return false; als letzte Anweisung (oder ein else-Zweig).

c) z. B. ein Held mit 100 Lebenspunkten und Stärke 10 ohne Waffe gegen ein Monster mit 50 Lebenspunkten und Stärke 20: angriffswert() = 10 < 50, aber 100 > 2 · 20 = 40 → true.

```
public static int beuteWert(Gegenstand[] schatz)
public static double verteileGold(int gold, int anzahl)
public static DynArray gegnerListe(Queue welle)
public static boolean istBesiegbar(Monster boss, Held held)
```

```
public static boolean istBesiegbar(Monster boss, Held held) {
    if (held.angriffswert() >= boss.getLeben()) {
        return true;
    } else if (held.getLeben() > 2 * boss.getStaerke()) {
        return true;
    }
    return false;
}
```

W6-04 Der Rechenfehler des Orakels · Fehler im Java-Code

a) Nicht akzeptiert: Zeile 6 – ein Zeichen (char) lässt sich nicht mit einer Zeichenkette (String) vergleichen; richtig c == '#' (einfache Hochkommas). Zeile 7 – getItem liefert Object; richtig int erster = (Integer) schaden.getItem(0);. Alle anderen Zeilen werden akzeptiert.

b) quote = 0.0 (7 / 8 ist ganzzahlig 0, erst danach wird in double umgewandelt) · quote2 = 87.5 (100.0 · 7 = 700.0 ist Kommazahl, also Kommazahl-Division) · mittel = 16 (49 / 3 ganzzahlig) · c = '#' · wand = true · erster = 12 · s = "Feld #1" (Verkettung von links: "Feld " + '#' → "Feld #", dann + 1) · code = 36 (char + int rechnet mit dem ASCII-Wert).

c) double quote = (double) treffer / versuche * 100; → 87.5. Der Cast muss vor der Division wirken; (double) (treffer / versuche) wäre zu spät (0.0).

```
2  double quote = (double) treffer / versuche * 100;   // 87.5
6  boolean wand = c == '#';                            // true
7  int erster = (Integer) schaden.getItem(0);          // 12
```

W6-05 Das Siegel der Ritter · Fehler im Java-Code

a) startsWith gehört nicht zu den erlaubten Zeichenketten-Operationen (erlaubt: Länge, Zeichen an Position, Teilzeichenkette, Verketten, Gleichheit, enthält, lexikographischer Vergleich, ASCII-Wert ↔ Zeichen). In istGast vergleicht g == name zwei Referenzen und nicht die Inhalte.

b) Die Online-IDE vergleicht Zeichenketten mit == inhaltlich – anders als Java und das Abitur. In Java ist == nur wahr, wenn beide Variablen auf dasselbe String-Objekt verweisen; ein zur Laufzeit zusammengesetzter Name (z. B. "Sir " + vorname) ist ein neues Objekt, == liefert dann false, obwohl der Text gleich ist. Gleichheit von Zeichenketten heißt im Abitur immer equals.

c) siehe Code. anzahlMitTitel liefert 2 ("Sir Aldo", "Sir Cato"): "Sira" beginnt mit "Sira" ≠ "Sir ", "sir Bert" scheitert an der Kleinschreibung. "Si" ist kürzer als der Titel: namen[i].substring(0, 4) löst bei "Si" eine StringIndexOutOfBoundsException aus – deshalb zuerst die Länge prüfen (&& wertet von links aus und bricht ab).

```
public static int anzahlMitTitel(String[] namen, String titel) {
    int anzahl = 0;
    for (int i = 0; i < namen.length; i++) {
        String n = namen[i];
        if (n.length() >= titel.length()
                && n.substring(0, titel.length()).equals(titel)) {
            anzahl++;
        }
    }
    return anzahl;
}
```

```
public static boolean istGast(String[] gaeste, String name) {
    for (String g : gaeste) {
        if (g.equals(name)) {
            return true;
        }
    }
    return false;
}
```

W6-06 Die Schatzsuche endet zu früh · Tracetabelle

a) siehe Tabelle: Nach drei Durchläufen ist lo = hi = 8; die Bedingung lo < hi ist falsch, die Schleife endet, ohne r[8] = 63 zu prüfen. Rückgabe −1, obwohl 63 in r steht.

b) Die Schleifenbedingung muss lo <= hi lauten: Auch ein Bereich aus genau einem Element (lo = hi) muss noch geprüft werden. Die fehlerhafte Methode findet 4, 22, 38 und 63 nicht – genau die Werte, bei denen sich der Suchbereich bis auf ein einziges Element verkleinert.

c) 9 Werte: höchstens 4 Durchläufe; 1000 Werte: höchstens 10 Durchläufe (jeder Durchlauf halbiert den Bereich, 2^10 = 1024 > 1000). Verdopplung der Anzahl → ein Durchlauf mehr.

```
Durchlauf | lo | hi | mid | r[mid]
----------+----+----+-----+-------
vorher    |  0 |  8 |  –  |   –
    1     |  5 |  8 |  4  |  31
    2     |  7 |  8 |  6  |  47
    3     |  8 |  8 |  7  |  56
Rückgabe: -1
```

```
while (lo <= hi) {   // korrigierte Zeile 4
```

W6-07 Das Muster des Drachen · Tracetabelle

a) siehe Tabelle; Ergebnis "--**". Die Rückgabewerte entstehen von unten nach oben: Erst wenn muster(0) mit "" zurückkehrt, kann muster(1) sein Ergebnis bilden usw.

b) Nur muster(0) ändert sich: Für n = 0 greift der neue Basisfall nicht, 0 ist gerade, also wird "-" + muster(0) berechnet – derselbe Aufruf ohne Fortschritt → Endlosrekursion, Abbruch mit StackOverflowError. Alle n ≥ 1 enden wie vorher bei muster(1) = "*" (gerade n ≥ 2 werden halbiert, ungerade n ≥ 3 um 1 verringert – 0 wird nie erreicht).

c) muster(6) = "-*": Das Ergebnis des rekursiven Aufrufs muster(n − 1) wird berechnet, aber nicht verwendet; jeder ungerade Aufruf liefert nur "*". Merke: Das Ergebnis des Selbstaufrufs muss in die Rückgabe eingehen.

d) Tiefe 5: 16 → 8 → 4 → 2 → 1 → 0 (Tiefen 0 bis 5). 16 = 2^4 wird viermal halbiert bis 1, dann folgt der Aufruf muster(0) als Basisfall – jeder Selbstaufruf erhöht die Tiefe um 1, der Erstaufruf hat Tiefe 0.

```
Rekursionstiefe | Aufruf    | Rückgabewert
----------------+-----------+-------------
       0        | muster(6) | "--**"
       1        | muster(3) | "-**"
       2        | muster(2) | "-*"
       3        | muster(1) | "*"
       4        | muster(0) | ""
```

W6-08 Das Protokoll des Kerkermeisters · Tracetabelle

a) z. B.: (1) Die Durchläufe mit x = 8 und x = 6 fehlen – dort ändert sich s (8 bzw. 6 werden abgelegt), jede Wertänderung muss dokumentiert werden. (2) Es fehlt die Zeile mit den Anfangswerten (q = 5, 8, 3, 6, s leer, summe = 0) und die vorgegebene Spalte Durchlauf. (3) Leere Zellen statt „leer“ bzw. „–“ – man kann nicht unterscheiden, ob die Struktur leer ist oder der Eintrag vergessen wurde. (4) Der Endwert von summe ist nicht ausdrücklich angegeben.

b) siehe Tabelle: 6 Durchläufe; summe = 24. Achtung: In Durchlauf 3 wird 8 vom Stapel genommen, um 1 erhöht und als 9 hinten in die Schlange eingereiht – die Schlange wird dabei wieder länger.

```
Durchlauf | x | q (vorn zuerst) | s (oben zuerst) | summe
----------+---+-----------------+-----------------+------
vorher    | – | 5, 8, 3, 6      | leer            |   0
    1     | 5 | 8, 3, 6         | leer            |   5
    2     | 8 | 3, 6            | 8               |   5
    3     | 3 | 6, 9            | leer            |   8
    4     | 6 | 9               | 6               |   8
    5     | 9 | 7               | leer            |  17
    6     | 7 | leer            | leer            |  24
summe nach der Schleife: 24
```

W6-09 Der Bauplan mit Java-Resten · Fehler im Struktogramm

a) z. B.: (1) Kopf ohne Typen: Signatur mit Parametertypen und Rückgabetyp angeben. (2) int anzahl = 0; ist eine Java-Deklaration – Zuweisung mit ← und ohne Typ und Semikolon: anzahl ← 0. (3) Schleifenkopf in Java-Syntax – Zählschleife als „für i von 0 bis d.getLength() − 1“ (oder „für jedes m in d“). (4) m = d.getItem(i): Zuweisung mit ←. (5) anzahl++ ist Java – anzahl ← anzahl + 1. (6) return anzahl; – „gib anzahl zurück“. (7) Semikolons gehören nicht ins Struktogramm.

b) i <= d.getLength(): Die Zählschleife läuft bis d.getLength() einschließlich; der letzte gültige Index ist aber d.getLength() − 1. Im letzten Durchlauf wäre d.getItem(d.getLength()) ein Zugriff auf einen nicht existierenden Index (Laufzeitfehler, P1).

c) siehe Struktogramm. Rückgabe 2 (12 und 7 sind kleiner als 30).

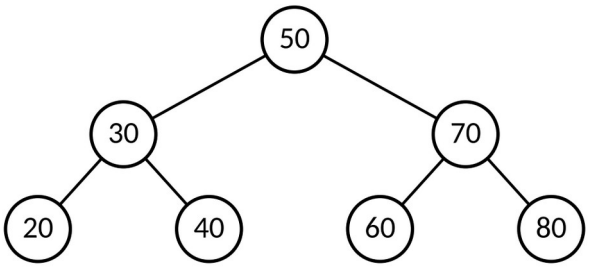
```
anzahlSchwach(d: dynamische Reihung vom Inhaltstyp Monster,
              grenze: Ganzzahl): Ganzzahl
  anzahl ← 0
  für i von 0 bis d.getLength() − 1
    m ← d.getItem(i)
    falls m.getLeben() < grenze
      dann: anzahl ← anzahl + 1
      sonst: –
  gib anzahl zurück
```

W6-10 Behaupten ist nicht begründen · Funktionsweise erläutern

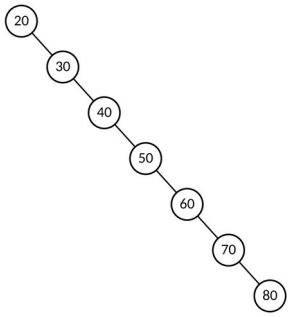
a) siehe Bäume: A ist vollständig mit Höhe 3, B ist eine rechte Kette mit Höhe 7.

b) „erläutern“ verlangt den nachvollziehbaren Weg (welche Vergleiche, welche Richtung), „begründen“ die Rückführung auf eine Gesetzmäßigkeit (ein Vergleich je Ebene, Höhe des Baums), „beurteilen“ ein Urteil mit Kriterien und Bezug zur Situation. Die Antwort nennt nur Ergebnisse („rechts an die 40“, „ausgeglichen“, „besser“) – ohne Weg, ohne Begründung, ohne Zahlen. Das reicht höchstens für die AFB-I-Anteile.

c) Einfügen von 45: Vergleich mit der Wurzel 50 – 45 ist kleiner, weiter links; Vergleich mit 30 – größer, weiter rechts; Vergleich mit 40 – größer; der rechte Teilbaum von 40 ist leer, also wird 45 dort als neues Blatt eingefügt (Ebene 3). Begründung: Eine Suche vergleicht auf jeder Ebene genau einmal und steigt dann eine Ebene ab; die Anzahl der Vergleiche ist also höchstens die Höhe. A hat drei Ebenen (Höhe 3: 1 + 2 + 4 = 7 Knoten), B ist entartet: Jede neue Nummer ist größer als alle vorhandenen und wird ganz rechts angehängt – sieben Ebenen, Suche nach 80 braucht 7 Vergleiche. Beurteilung: Bei 1000 Truhen braucht ein ausgeglichener Baum höchstens 10 Vergleiche (Höhe 10, da 2^10 − 1 = 1023 ≥ 1000), die sortierte Einfügereihenfolge erzeugt eine Kette mit bis zu 1000 Vergleichen – wie eine lineare Suche. Bei Verdopplung der Truhenzahl kommt beim ausgeglichenen Baum nur ein Vergleich hinzu, bei der Kette verdoppelt sich der Aufwand. Eine Einfügereihenfolge, die einen ausgeglichenen Baum erzeugt (Mitte zuerst), ist daher deutlich geeigneter.



Baum A (Höhe 3)



Baum B (Höhe 7)

W6-11 Zwei Namen, ein Held · Objektkarten und Referenzen

a) Es gibt drei Objekte: held1: Held (name = "Runa", leben = 50, maxLeben = 80, staerke = 12, gold = 0, waffe → waffe1) – darauf verweisen h und k; waffe1: Waffe (name = "Drachenklinge", wert = 120, schaden = 18) – darauf verweisen w und das Attribut waffe von held1; held2: Held (name = "Runa", leben = 80, maxLeben = 80, staerke = 12, gold = 0, waffe = null) – darauf verweist t. Zeile 3 erzeugt kein neues Objekt, sondern kopiert nur die Referenz.

b) gleich1 = true, gleich2 = false, a = 30 (12 + 18), b = 12 (keine Waffe).

c) == vergleicht bei Objekten die Referenzen, also die Identität: h und k sind zwei Referenzen auf dasselbe Objekt held1 – deshalb wirken ausruesten über k und nimmSchaden über h auf dasselbe Objekt, und h == k ist wahr. t verweist auf ein zweites, neu erzeugtes Objekt held2; obwohl es mit denselben Werten erzeugt wurde (und sich inzwischen sogar unterscheidet), ist es ein anderes Objekt – h == t ist falsch. Ein inhaltlicher Vergleich müsste die Attribute über Getter vergleichen (bei Zeichenketten equals, P5).

```
h, k ──▶ held1: Held                  w ──▶ waffe1: Waffe
         name = "Runa"                      name = "Drachenklinge"
         leben = 50                         wert = 120
         maxLeben = 80                      schaden = 18
         staerke = 12                            ▲
         gold = 0                                │
         waffe ──────────────────────────────────┘
```

```
t ──────▶ held2: Held
          name = "Runa", leben = 80, maxLeben = 80,
          staerke = 12, gold = 0, waffe = null
```

W6-12 Das Sortier-Duell der Teams · Funktionsweise erläutern

a) siehe Tabelle; Rückgabe 10. In jedem Durchlauf wird das kleinste Element des unsortierten Rests gesucht und mit r[i] vertauscht.

b) Im Durchlauf i wird mit allen Elementen rechts von i verglichen: (n − 1) + (n − 2) + … + 1 = n(n − 1)/2 Vergleiche – unabhängig von der Anordnung. n = 1000: 499 500; n = 2000: 1 999 000 – etwa das Vierfache, denn (2n)(2n − 1)/2 ≈ 4 · n(n − 1)/2 (quadratisches Wachstum). Abschätzen heißt: Formel ansetzen, Faktor angeben, mit Zahlen belegen.

c) j <= r.length: Schon im ersten Durchlauf erreicht j den Wert 5, der Zugriff r[5] löst eine ArrayIndexOutOfBoundsException aus (gültig: 0 bis r.length − 1). i < r.length dagegen ist harmlos: Im zusätzlichen letzten Durchlauf (i = 4) läuft die innere Schleife nicht (j beginnt bei 5), r[4] wird mit sich selbst vertauscht – gleiche Ergebnisse, gleiche 10 Vergleiche, nur ein überflüssiger Durchlauf. Nicht jedes Off-by-one führt zum Absturz – prüfe Grenzen deshalb immer am letzten Durchlauf.

```
nach Durchlauf | r
---------------+--------------------
   i = 0       | 3, 14, 27, 9, 20
   i = 1       | 3, 9, 27, 14, 20
   i = 2       | 3, 9, 14, 27, 20
   i = 3       | 3, 9, 14, 20, 27
Rückgabe (vergleiche): 10
```

W6-13 Die Einzelgänger im Truhenbaum · Methode implementieren

a) siehe Code. einKind(S) = 3: Die Knoten 35 (nur links 29), 58 (nur rechts 61) und 90 (nur rechts 95) haben genau einen nicht leeren Teilbaum. Basisfall ist der leere Baum; das Ergebnis eines Knotens = Ergebnis links + Ergebnis rechts + 1, falls genau ein Teilbaum leer ist.

b) Aufrufreihenfolge: einKind(41) → einKind(23) → einKind(12) (Blatt, 0) → einKind(35) → einKind(29) (Blatt, 0) → einKind(rechter Teilbaum von 35). Dieser Teilbaum ist leer: isLeaf() auf einem leeren Baum löst eine IllegalStateException aus. Ursache: Der Basisfall „Blatt“ deckt Knoten mit genau einem Kind nicht ab – deren leerer Teilbaum wird trotzdem aufgerufen (P1). Basisfall ist immer der leere Baum.

c) Ergebnis 0: Die Wurzel 41 hat zwei Kinder, zaehler bleibt 0. Die Ergebnisse der rekursiven Aufrufe werden berechnet, aber verworfen (P7) – sie müssen zu zaehler addiert werden. (Der Vergleich zweier Wahrheitswerte mit != ist übrigens korrekt: wahr, wenn genau einer der beiden Teilbäume leer ist.)

```
public static int einKind(BinTree b) {
    if (b.isEmpty()) {
        return 0;
    }
    int anzahl = einKind(b.getLeft()) + einKind(b.getRight());
    boolean linksLeer = b.getLeft().isEmpty();
    boolean rechtsLeer = b.getRight().isEmpty();
    if ((linksLeer && !rechtsLeer) || (!linksLeer && rechtsLeer)) {
        anzahl = anzahl + 1;
    }
    return anzahl;
}
```

W6-14 Fallen im Thronsaal · Verändern und Erweitern

a) (0, 0) → 0 · (1, 3) → 2 (Fallen auf (0, 2) und (1, 2)) · (2, 1) → 3 (Fallen auf (1, 2), (2, 0) und (3, 1)) · (3, 5) → 2 (Fallen auf (2, 5) und (3, 4)).

b) (0, 0) und (3, 5) liegen am Rand: Zugriffe wie karte[−1][−1] bzw. karte[4][6] lösen eine ArrayIndexOutOfBoundsException aus (gültig sind Zeilen 0 bis karte.length − 1 und Spalten 0 bis karte[0].length − 1). (1, 3) liefert 3 statt 2, weil das Feld selbst eine Falle ist und bei dz = 0, ds = 0 mitgezählt wird. (2, 1) liefert zufällig das richtige Ergebnis 3 – das Feld selbst ist frei.

c) siehe Code: Für jeden Nachbarn zuerst prüfen, ob er im Feld liegt (Zeile und Spalte jeweils ≥ 0 und kleiner als die Länge), und das Feld selbst auslassen. Die Reihenfolge in der Bedingung ist wichtig: && wertet von links aus, der Zugriff steht deshalb am Ende.

```
public static int fallenRingsum(char[][] karte, int z, int s) {
    int anzahl = 0;
    for (int dz = -1; dz <= 1; dz++) {
        for (int ds = -1; ds <= 1; ds++) {
            int nz = z + dz;
            int ns = s + ds;
            if (!(dz == 0 && ds == 0)
                    && nz >= 0 && nz < karte.length
                    && ns >= 0 && ns < karte[0].length
                    && karte[nz][ns] == '^') {
                anzahl++;
            }
        }
    }
    return anzahl;
}
```